The Novendor blueprint: running an AI consulting firm from your phone

OpenClaw can power Novendor as a fully autonomous AI consulting operation using 10 specialized agents, Lobster deterministic pipelines, and Telegram as the command interface — but the agents you build first should generate revenue, not write code. The critical insight from this research: solo AI consultancies that prioritize outreach, content, and proposal automation before technical delivery infrastructure reach profitability 3-4x faster. The $11B AI consulting market (projected >$90B by 2035) Articsledge has a massive gap — **91% of mid-market companies now use generative AI, yet 42% abandon initiatives before production** because they lack structured guidance. Novendor's secret weapon is dogfooding: using the very multi-agent system you sell to run the sales process itself. Wikipedia What follows is the complete configuration blueprint — from exact OpenClaw JSON to Lobster YAML pipelines to cold email sequences — organized around a three-phase rollout that gets clients in the door while building delivery infrastructure in parallel.

The 10-agent architecture running on one gateway

OpenClaw runs a single Gateway daemon Milvus (default ws://127.0.0.1:18789) GitHub that hosts multiple fully isolated agents. Medium Each agent gets its own workspace, Zenvanriel session store, auth profiles, and model configuration under ~/.openclaw/agents/<agentId>/. OpenClaw openclaw Messages route to agents through deterministic bindings matched on channel, account, peer, or group topic. GitHub +3 **Lobster — OpenClaw's YAML workflow engine — handles all inter-agent orchestration deterministically**, Openclaws replacing unreliable LLM-controlled flow with typed pipelines that support loops, approval gates, and structured data passing. DEV Community OpenClaw

The architecture splits into two functional halves connected through the Commander agent:

```
REVENUE ENGINE                    DELIVERY ENGINE
Outreach → Proposal Writer        Architect → Builder → QA
Content/Marketing ↗   ↘              ↑          ↑
Demo Generator        Commander ←→ Operations ←← feedback loop
```

Here is the core gateway configuration for all 10 agents:

```
json5
```

```
{
  agents: {
    defaults: {
      model: "anthropic/claude-sonnet-4-5",
      heartbeat: { every: "30m", target: "last" },
      subagents: {
        maxConcurrent: 8,
        maxSpawnDepth: 2,
        runTimeoutSeconds: 900,
        model: "anthropic/claude-sonnet-4-5"
      }
    },
    list: [
      {
        id: "commander", default: true, name: "Commander",
        workspace: "~/.openclaw/workspace-commander",
        model: "anthropic/claude-opus-4-6",
        identity: { name: "Commander", emoji: "🎯" },
        tools: {
          allow: ["exec", "read", "write", "sessions_list", "sessions_send",
                  "sessions_spawn", "session_status", "lobster", "web_search", "web_fetch", "cron"]
        },
        subagents: {
          allowAgents: ["architect", "builder", "qa", "sales", "proposals",
                        "content", "demo", "operations", "data"]
        }
      },
      {
        id: "architect", name: "Architect",
        workspace: "~/.openclaw/workspace-architect",
        model: "anthropic/claude-opus-4-6",
        identity: { name: "Architect", emoji: "🏗" },
        tools: { allow: ["read", "write", "exec", "web_search", "web_fetch"], deny: ["browser", "canvas"] }
      },
      {
        id: "builder", name: "Builder",
        workspace: "~/.openclaw/workspace-builder",
        model: "anthropic/claude-sonnet-4-5",
        identity: { name: "Builder", emoji: "🔨" },
        sandbox: { mode: "all", scope: "agent", docker: { network: "bridge" } },
        tools: { allow: ["exec", "read", "write", "edit", "apply_patch"], deny: ["browser", "canvas"] }
      },
      {
```

```
    id: "qa", name: "QA",
    workspace: "~/.openclaw/workspace-qa",
    identity: { name: "QA", emoji: "🔍" },
    tools: { allow: ["exec", "read", "web_fetch"], deny: ["write", "edit", "browser"] }
  },
  {
    id: "data", name: "Data",
    workspace: "~/.openclaw/workspace-data",
    identity: { name: "Data", emoji: "📊" },
    tools: { allow: ["exec", "read", "write", "web_search"], deny: ["browser"] }
  },
  {
    id: "sales", name: "Outreach",
    workspace: "~/.openclaw/workspace-sales",
    identity: { name: "Outreach", emoji: "💼" },
    tools: { allow: ["read", "write", "web_search", "web_fetch"], deny: ["exec", "browser"] }
  },
  {
    id: "proposals", name: "Proposal Writer",
    workspace: "~/.openclaw/workspace-proposals",
    identity: { name: "Proposals", emoji: "📝" },
    tools: { allow: ["read", "write", "web_search", "web_fetch"], deny: ["exec", "browser"] }
  },
  {
    id: "content", name: "Content",
    workspace: "~/.openclaw/workspace-content",
    identity: { name: "Content", emoji: "✍️" },
    tools: { allow: ["read", "write", "web_search", "web_fetch"], deny: ["exec", "browser"] }
  },
  {
    id: "demo", name: "Demo Generator",
    workspace: "~/.openclaw/workspace-demo",
    identity: { name: "Demo", emoji: "🎬" },
    tools: { allow: ["read", "write", "exec", "web_fetch"], deny: ["browser"] }
  },
  {
    id: "operations", name: "Operations",
    workspace: "~/.openclaw/workspace-operations",
    identity: { name: "Ops", emoji: "⚙️" },
    tools: { allow: ["read", "write", "exec", "cron"], deny: ["browser"] }
  }
]
```

```
  }
}
```

Critical configuration gotchas discovered by the community: never reuse `agentDir` across agents (causes session collisions), OpenClaw GitHub put `allowAgents` in the per-agent config not in `agents.defaults` (crashes gateway on startup), GitHub and always explicitly set `sandbox.docker.network: "bridge"` since Docker containers default to `network: 'none'` which silently breaks web tools. Getopenclaw

---

Telegram as the nerve center of operations

The optimal Telegram architecture uses a **Forum Supergroup** with topic-based agent routing, supplemented by separate bots per agent for DM-based interaction. Forum topics act like tabs on a phone screen openclaw — Paul swipes between Research, Builds, Client Ops, and Status to see each agent's latest output at a glance.

Novendor HQ Forum Supergroup layout:

Topic	Agent	Behavior
📢 General	Commander	`requireMention: true` — responds only when asked
🔬 Research	Architect	Responds freely — all research output lands here
🏗️ Builds	Builder + QA	Build status, test results, deploy confirmations
👤 Client Ops	Proposals + Operations	Proposals, invoicing, project tracking
💼 Sales	Outreach + Content	Lead gen reports, content drafts, campaign stats
📊 Status	All (read-only)	Heartbeat alerts, automated status summaries

Each topic gets its own session key (`agent:research:telegram:group:<id>:topic:3`), providing complete memory isolation. openclaw stack-junkie The Commander agent runs on a separate bot bound to DMs for private commands, while specialist agents respond freely in their dedicated topics. OpenClaw's **heartbeat system** checks each agent's `HEARTBEAT.md` OpenClaw every 30 minutes but only messages Paul when something needs attention Milvus DEV Community — preventing notification fatigue.

The command vocabulary for phone-driven operations:

```
/research [topic]          → Spawns async research sub-agent, reports to 🔬
/propose [client] [scope]  → Generates proposal, sends to 👤 for review
/build [feature]           → Dispatches Lobster pipeline, updates 🏗️
/outreach [target]         → Triggers prospect research + email draft
/content [topic]           → Generates LinkedIn post draft to 💼
/status                    → All-agent status summary to 📊
/approve [id]              → Tappable inline button for pending items
/brief                     → Morning briefing (cron-triggered at 7am)
/cost [project]            → Token spend summary
```

The morning briefing arrives automatically via cron at 7am: inbox priorities, today's calendar, overdue tasks, pending approvals, and pipeline status — all in a compact, emoji-prefixed format designed for a phone screen. **Inline keyboard buttons** handle approvals with a single tap: Telegram [✅ Approve] [❌ Deny] [✏️ Revise]. OpenClaw's execApprovals config ensures destructive operations (deploying code, sending client emails, processing invoices) always pause for human confirmation. GitHub +2

Voice messages work natively — OpenClaw transcribes audio through Whisper, so Paul can dispatch work while walking. The design principle: **phone for command and control, desktop for deep work**. Dan-malone dan-malone Fire-and-forget commands from the phone; agents work autonomously for 10-60 minutes; results arrive as notifications.

Lobster pipelines make agent coordination deterministic

The community consensus — crystallized in Gustavo Gondim's influential contribution to Lobster — is that **LLMs should never control workflow routing**. DEV Community Every time flow control lives in a prompt, it introduces a failure mode. dev DEV Community Lobster replaces this with typed YAML pipelines where LLMs handle the creative work and deterministic code handles the plumbing. DEV Community +2

The core development pipeline (commander → architect → builder → QA with retry loops):

```yaml
```

```yaml
name: novendor-dev-pipeline
args:
  project: {}
  task: {}
  chat_id: {}

steps:
  - id: architect-design
    command: >
      openclaw.invoke --tool agent-send --args-json '{
        "agentId": "architect",
        "message": "Design architecture for: ${task}. Project: ${project}. Return JSON with specs, file list, and acceptance crite
        "sessionKey": "pipeline:${project}:architect"
      }'

  - id: build-and-test
    lobster: ./build-review.lobster
    args:
      project: ${project}
      specs: $architect-design.stdout
    loop:
      maxIterations: 3
      condition: '! echo "$LOBSTER_LOOP_JSON" | jq -e ".passed" > /dev/null'

  - id: notify-success
    command: >
      openclaw.invoke --tool message --action send --args-json '{
        "provider": "telegram",
        "to": "${chat_id}",
        "text": "✅ ${project}: pipeline complete. QA passed on iteration $LOBSTER_LOOP_ITERATION."
      }'
    condition: $build-and-test.json.passed == true
```

The `build-review.lobster` sub-workflow sends work to the Builder agent, passes output to QA, and returns a structured `{passed: boolean, feedback: string}` JSON. If QA fails, the loop re-invokes Builder with the feedback — up to 3 iterations. Session keys follow the pattern `pipeline:<project>:<role>`, giving project isolation, role separation, and addressability in a single string. dev

Why Lobster over raw sub-agents: sub-agents have a spawn depth limit of 2, Openclaw Wiki Docs use non-deterministic LLM flow control, and generate UUID-based session keys Openclaw Wiki Docs that can't be addressed by project name. DEV Community Lobster has none of these limitations, supports approval gates with

resume tokens, LobeHub and produces auditable YAML that can be diffed, logged, and replayed. Learn OpenClaw

The proposal generation pipeline (business-side):

```yaml
name: novendor-proposal-pipeline
args:
  client: {}
  scope: {}
steps:
  - id: prospect-research
    command: >
      openclaw.invoke --tool agent-send --args-json '{
        "agentId": "sales",
        "message": "Research ${client}: company size, industry, recent AI initiatives, pain points, key decision-makers. Return s
        "sessionKey": "pipeline:${client}:research"
      }'

  - id: generate-proposal
    command: >
      openclaw.invoke --tool agent-send --args-json '{
        "agentId": "proposals",
        "message": "Generate proposal for ${client}. Scope: ${scope}. Research: $prospect-research.stdout. Use template from c
        "sessionKey": "pipeline:${client}:proposal"
      }'
    stdin: $prospect-research.stdout

  - id: send-for-approval
    command: >
      openclaw.invoke --tool message --action send --args-json '{
        "provider": "telegram",
        "to": "${chat_id}",
        "text": "📝 Proposal ready for ${client}:\n$generate-proposal.stdout\n\nGenerated in $generate-proposal.durationMs ms
      }'
    approval: required
```

The approval gate pauses the pipeline until Paul taps Approve in Telegram, at which point it resumes LobeHub and sends the proposal to the client. Learn OpenClaw

---

Claude CLI and Codex CLI as the coding engines

The Architect agent invokes **Claude CLI** for reasoning-heavy tasks, while the Builder agent invokes **Codex CLI** for code generation. The task routing reflects fundamental architectural differences: Claude uses **3-4x more tokens but produces more thorough, deterministic output**, Morph while Codex is faster with lower token usage but more variable between runs. Composio

Task routing matrix:

Task	Tool	Rationale
Architecture analysis	Claude CLI (`-p --output-format json`)	Superior long-form reasoning, 200K+ context
Code review + security audit	Claude CLI	More true positives in vulnerability detection
Research and investigation	Claude CLI	WebSearch/WebFetch tools, deep codebase understanding
Code generation	Codex CLI (`exec --full-auto --json`)	Fast output, efficient tokens, direct file operations
File editing and refactoring	Codex CLI	Strong sandbox model, workspace-write mode
Test writing	Codex CLI	Efficient at mechanical code generation
Bug fixing	Codex CLI	Leads Terminal-Bench 2.0 at 77.3% vs 65.4% Morph

Architect shell invocation pattern:

```bash
claude -p "Analyze task: '$TASK_SPEC'. Produce implementation plan with file-by-file changes, API contracts, and data mod
  --output-format json \
  --allowedTools "Read,Grep,Glob,WebSearch" \
  --append-system-prompt "You are Novendor's senior architect." \
  --max-turns 15 \
  --max-budget-usd 2.00 \
  > /tmp/architecture-plan.json
```

Builder shell invocation pattern:

```bash
codex exec --full-auto --model gpt-5.4 \
  "Read /tmp/architecture-plan.json and implement exactly as specified." \
  --json > /tmp/build-log.jsonl
```

QA validation with structured output:

```bash
claude -p "Run 'npm test' and analyze results against the spec." \
  --output-format json \
  --json-schema '{"type":"object","properties":{"passed":{"type":"boolean"},"failures":{"type":"array","items":{"type":"string
  --allowedTools "Bash(npm test:*),Read,Grep"
```

OpenClaw's `exec` tool gives agents shell access — configure it with `security: "allowlist"` on the gateway host and `security: "full"` only inside sandboxed Docker containers. Getopenclaw DeepWiki Both CLIs support **session resumption** (Claude via `--resume <session_id>`, 32blog Codex via `resume --last`), Onlinetool enabling multi-turn pipelines where QA feedback feeds back into the architect's session for iterative refinement. Claude

---

The git-based research-to-implementation pipeline

Deep research outputs from ChatGPT, Claude, and Codex land in `docs/research/` with standardized YAML frontmatter (title, date, source, priority, status tags). This follows the **Spec-Driven Development** pattern now endorsed by GitHub's Spec Kit: research → spec → plan → tasks → code. Microsoft Developer

Repository structure:

```
novendor/
├── docs/
│   ├── research/        # Deep research reports (ChatGPT/Claude/Codex)
│   │   └── 2026-03-12-topic-name.md  (frontmatter: status, priority, tags)
│   ├── specs/           # Feature specifications generated from research
│   │   ├── spec.md      # PRD with user stories and requirements
│   │   ├── plan.md      # Technical plan with architecture decisions
│   │   └── tasks.md     # Atomic implementation tasks
│   ├── architecture/    # ADRs and system diagrams
│   └── proposals/       # Client-facing proposals with templates
├── src/                 # Implementation code
├── tests/               # Test suites
├── .openclaw/           # Agent workspace configs
└── memory/              # Shared knowledge graph (memory.jsonl)
```

The trigger chain: a `post-commit` git hook detects new files in `docs/research/`, fires a webhook to the OpenClaw gateway, which dispatches the Commander agent to triage. [OpenClaw] Commander reads the frontmatter, assigns priority, and triggers the Architect via Lobster pipeline to generate specs. Each stage transition is a git commit with a conventional prefix (`research:`, `spec:`, `arch:`, `impl:`, `qa:`). Status tracking lives in the frontmatter itself — `pending-review → spec-generated → tasks-generated → implementing → complete` — making the pipeline state visible in any git diff.

Branch strategy: `main` is production-ready and human-approved. Feature work branches as `impl/task-001`, consolidates into `review/feature-name` PRs. **Human review gates exist at spec approval and PR merge** — agents never merge to main autonomously. This matches the GitHub Agentic Workflows principle: "pull requests are never merged automatically." [GitHub] [GitHub]

Revenue engine: how Novendor gets its first clients

The AI consulting market has an enormous gap at the mid-market level. Big 4 firms charge **$500K-$3M for strategy phases alone** and deliver PowerPoint decks. [Innovative Human Capital] Mid-size companies allocate **$85,521/month on AI on average** (up 36% from 2024) but lack the internal expertise to spend it effectively. Novendor's positioning: **"We eat our own dog food"** [Wikipedia] — running the entire consulting business on the same AI agent infrastructure being sold to clients.

The five-channel acquisition strategy, in priority order:

LinkedIn thought leadership is the highest-leverage channel. Only 1.1% of LinkedIn users post weekly, creating a massive opportunity gap. [FlowHunt] The content strategy mixes behind-the-scenes glimpses of the multi-agent system in action ("our AI generated this proposal in 20 minutes"), frameworks for confused

executives ("The 3 Questions Every $200M Company Should Ask Before Hiring an AI Consultant"), and contrarian takes challenging AI hype. Jeffbullas **Post 3-5 times weekly** with a consistent editorial voice.

Cold email sequences with AI-personalization deliver the highest volume. Marketowl A 5-touch sequence over 14 days starts with a pain-identification email, escalates through value delivery and social proof, and closes with a direct ask for 15 minutes. The Novendor edge: each email transparently reveals it was AI-researched. "*P.S. This email was researched and drafted by our AI system in 47 seconds. The insights are real. Imagine what we could do for [Company Name]'s operations.*"

The AI Readiness Assessment is the conversion engine — a 25-question self-assessment across data infrastructure, team skills, process maturity, tech stack, and strategic alignment that achieves **30-40% opt-in rates** on dedicated landing pages. digitalapplied Digital Applied After completion, OpenClaw auto-generates a personalized report with the prospect's maturity score benchmarked against industry peers, top 3 recommended AI use cases, and a 90-day implementation roadmap. Include a note: "*This analysis was generated by our AI system in under 60 seconds. A traditional firm would charge $25,000+ and take 4 weeks.*"

Network and referrals convert at the highest rate. Paul's existing REPE network from Winston is the most immediate pipeline. Strategic partnerships with accounting firms, IT managed services, and fractional CTO/CFO firms who get asked about AI but lack expertise create referral flows.

Content marketing compounds over time. Consulting Success® The "building in public" series documents using AI to run Novendor itself — every week becomes a case study. One article produces 7+ LinkedIn posts, 3 email sequences, and a lead magnet through AI repurposing.

Messaging that resonates with confused mid-size executives:

✅ **Language that works**	❌ **Language that repels**
"Working system in 6 weeks"	"Comprehensive roadmap"
"Your first AI win"	"Enterprise-wide AI strategy"
"We built this, let me show you"	"Our methodology framework"
"$50K investment → $500K annual savings"	"Future-proof your organization"

Pricing architecture: Tier 1 is a free AI Readiness Assessment (lead gen) or $5K-$15K paid deep assessment. Tier 2 is a $25K-$75K Quick Win pilot project (single use case, 4-8 weeks, working system in production). Tier 3 is a $10K-$25K/month Fractional AI Director retainer. Tier 4 is a $100K-$300K custom AI platform build (like Winston for their industry). Move toward **value-based and project-based pricing** — hourly billing punishes efficiency when AI makes you faster. Consulting Success®

Workspace personality: SOUL.md files that shape each agent

Every agent's behavior is defined by its SOUL.md — loaded at every session start. MMNTM Milvus These are not generic prompts; they encode deep operational knowledge.

Commander SOUL.md should establish: decisive orchestration tone, delegation-first behavior (never writes code or designs architecture directly), pipeline-aware routing rules, and cost-consciousness (track token spend, use cheaper models for sub-agent work). It should know Paul's communication preferences, working hours, and approval thresholds.

Outreach SOUL.md should encode: Novendor's ICP (mid-size companies, $50M-$500M revenue, confused about AI), the anti-Big-4 positioning, dasadvancedsystems specific language guidelines (use "practical AI that ships" not "digital transformation"), the 5-touch email sequence structure, LinkedIn connection request templates, and trigger events to watch for (funding announcements, CTO hires, competitor AI adoption).

Proposal Writer SOUL.md should include: three proposal templates (Assessment, Quick Win, Retainer), Winston as the lead case study, ROI calculation frameworks by industry, pricing guidelines, and the dogfooding narrative ("this proposal was generated by our AI system in 22 minutes").

Builder SOUL.md should specify: implement exactly what Architect specifies, never deviate without flagging, always run tests before reporting completion, use git worktrees for parallel work, and include structured output format for QA ingestion.

The shared memory/ directory contains a knowledge graph (memory.jsonl) with entity types including client, prospect, project, deliverable, decision, and lesson. All agents read and write to this common memory. OpenClaw An **Improver meta-agent** (Phase 3) periodically reads "lesson" entities and upgrades other agents' SOUL.md files — creating a self-improving system. DEV Community

---

The three-phase rollout: revenue first, delivery second, scale third

Phase 1: Revenue engine (weeks 1-2)

Goal: Start generating leads and pipeline. This is existential — no clients means no business.

Week 1 deliverables: Deploy Commander agent (basic orchestration). Deploy Content agent (begin LinkedIn posting immediately, 3-5 posts/week). Set up HubSpot CRM (free tier). Build AI Readiness Assessment landing page. Configure cold email infrastructure (domain warmup takes 2 weeks — start now).

Week 2 deliverables: Deploy Outreach agent with LinkedIn automation (Valley AI or Expandi, $99/month) and email sequences (Instantly.ai, $30/month). Deploy Proposal Writer agent with three templates. Configure PandaDoc for e-signatures. Heyiris Build initial prospect list of **500 mid-market companies** in target industries. Target: 50-100 outreach messages per week, 3-5 discovery calls per week.

Telegram setup for Phase 1: Create 3 bots via @BotFather (Commander, Sales, Content). OpenClaw Create Novendor HQ forum supergroup with General, Sales, and Status topics. dan-malone Configure morning briefing cron for 7am.

Phase 2: Delivery infrastructure (weeks 3-4)

Goal: Be able to deliver client work when the first contracts close.

Week 3 deliverables: Deploy Architect agent with Claude CLI integration. Deploy Builder agent with Codex CLI integration and Docker sandbox. Set up the git pipeline (docs/research/ → docs/specs/ → src/). Configure Lobster development pipeline (architect → builder → QA with retry loop).

Week 4 deliverables: Deploy QA agent with automated testing. Deploy Operations agent with Notion project tracking and Stripe invoicing. Set up inter-agent communication via shared knowledge graph. Create 2 more Telegram bots (Builder, Operations) and add Builds and Client Ops topics.

Phase 3: Scale and self-improvement (month 2+)

Goal: Operate like a 10-person firm with full automation.

Deploy Data agent for database design and RAG pipeline management. Deploy Demo Generator agent to create tailored demo library from Winston. Implement the Improver meta-agent that reads lessons and upgrades other agents. Build automated case study generation from completed projects. Scale outreach to 200+ messages per week. Target: **3-5 concurrent client projects, $20K+ monthly revenue, pipeline of 10-20 qualified prospects**.

The full system at maturity: Paul sends a voice message from his phone — "Research Acme Corp and generate a proposal for AI customer service" — and walks away. The Outreach agent researches Acme's 10-K, LinkedIn presence, and recent news. The Proposal Writer generates a personalized 15-page proposal with ROI estimates. It arrives in the Client Ops topic with [✅ Send] [✏️ Revise] buttons. Paul taps Approve over coffee. The proposal emails automatically. When Acme signs, Commander creates the project, Architect generates specs from the SOW, Builder implements, QA validates, and Operations tracks deliverables and invoices milestones. **The entire consulting firm runs from a Telegram chat.**

Conclusion: what makes this architecture work

Three design decisions separate this from a toy demo. First, **deterministic orchestration via Lobster** instead of LLM-controlled routing eliminates the class of failures where agents decide wrong about what to do next. DEV Community Creative work stays with the LLMs; plumbing stays in YAML. dev Second, **revenue agents before delivery agents** — the Outreach, Content, and Proposal Writer agents generate pipeline while the technical infrastructure catches up. Most technical founders make the opposite mistake. Third, **the dogfooding loop** Wikipedia where Novendor uses its own AI system to sell AI consulting creates an irresistible proof point. When the proposal itself demonstrates the capability, the sales cycle collapses.

The known risks are real: OpenClaw is pre-1.0 and ships multiple releases per week, Substack the agentToAgent + sessions_spawn interaction has a known bug (GitHub issue #5813), WhatsApp disconnects every few days, Substack and **approximately 2% of AI-generated code contains security vulnerabilities** per ICSE 2026 research. Augment Code Human review at spec approval and PR merge remains non-negotiable. GitHub GitHub But for a solo operator willing to stay close to the system, this architecture transforms one person into a consulting firm that operates at 10x the output of a traditional solo practice — and proves the value proposition with every client interaction.